California Polytechnic State University Pomona

DEPARTMENT OF ELECTRICAL & COMPUTER ENGINEERING

ECE 3301L.01

PROJECT REPORT: PONG

Prepared by

Andrew Cervantes, Thanarith Khep, Damaris Sanchez Meraz, Ivan Kao

Presented to

Mohammed Aly

May 11th, 2026

(DESCRIPTION OF SYSTEM EXPLANATION)

The overall project involves creating a Pong game system which was implemented on the Nexys A7, which ultimately has various features that include a VGA display output and UART communication.This project was able to implement various components that include the game logic, visuals, external controls that allowed the game to be played by using the FPGA's switches and buttons or remotely through the PC terminal via UART.

The build up with this project consists of provided design sources and custom implemented design sources. The UART transmitter and receiver modules were given and implemented into the design. There were only small modifications made to match the system of our clock frequency. These modules were able to enable the communication between the FPGA and the PC, which allowed the keyboard inputs such as A,D,S and P that ultimately controlled the game in real time. The overall design was able to demonstrate how handling, game logic and display systems could be overall combined into a single hardware-based application.

(MODULES EXPLANATION)

The main 7 design modules consist of a clock generator, the animation, the top module, the seven-segment controller, the VGA synchronization module , the implemented receiver and the transmitter.

The *"clk_50MHz_generator"* is responsible for generating and making sure that the game can be fully operational in terms of the speed and the overall timing. The clock defines the timing reference for synchronous logic within the system. It is able to make sure that there is a consistent game speed, animation updates and proper communication timing.

The *"pong_animate_unit"* module is what implements the core game logic. The logic implemented controls the movement of the paddle and ball. Aside from this it is able to detect the collision that occurs if the ball hits the paddle or if it misses. It will also update the position with every reset cycle. It also generates the graphics, including the paddle, ball, walls and the score display.

Our top module *"pong_top"* is what combines everything together and connects all the submodules. This includes the score system tracking, UART communication, the timing control and the LED indicators. It is able to score the updates which in our case has an increase of 5 points every time it has a collision with the paddle and resets when the ball is missed.

The "*seven_seg_controller"* module takes the game's score and displays it on the 8 digit 7 segment display. Because all eight digits share the same segment wires, only one digit can be driven at a time. The module handles this by using a free-running counter to rapidly cycle through each digit one at a time, pulling its anode low to turn it on and putting the correct segment pattern on seg[7:0] for that digit's value.

The "*vga_sync"* module takes in the 50 MHz clock and makes all the time signals that we need for the VGA output. It took in the two counters which tracked the horizontal pixel position and the row, which are used to generate *hsync* and *vsync* for the monitor. It outputs *pixel_x* and then *pixel_y* which indicates the pixel that was being drawn. Alongside this we have *video_on* in order to show the display within the 640x480 area. The 25 MHz p_tick was generated for the timing.

The *"transmitter"* module sends one byte at a time over the *uart_tx* to the computer which uses the five states, **Idle**, **Start**, **Data Bits**, **Stop Bit**, and **Refresh**. When *pong_top* wants to send a character, it puts the byte on *i_Byte* and pulses *i_DV* for one cycle. This then triggers the FSM and each bit is held for 434 clock cycles. During this *o_Sig_Active* stays high so *pong_top* knows not to load a new byte yet. The transmitter is used for echoing acknowledgements back to the PC for the valid UART command that is received (A, D, S, or P). Then *pong_top* responds by sending back a single confirmation character (L, R, S, or P). If a new command arrives while the transmitter is busy, *pong_top* queues the echo in an *echo_pending* register and sends it as soon as *o_Sig_Active* goes low.

The *"receiver"* module listens on the *uart_rx* for incoming bytes from the computer. It uses the same five-state FSM structure as the transmitter: **Idle**, **Start**, **Data Bits**, **Stop Bit**, and **Refresh**. When the line goes low, the FSM wakes up and waits half a bit period before sampling, which lands it in the middle of the start bit rather than the edge. After all the 8 data bits and the stop bit are received o_DV is asserted for one clock cycle and the received byte is output on

o_Byte for **pong_top** to read it. The module also includes a two flip flop synchronizer on the RX input before it reaches the FSM. Since the incoming serial data arrives asynchronously from the PC and is not aligned to the FPGA's clock, sampling it directly could cause metastability issues.Each valid byte is then interpreted by the top module as a game command such as A, D, S, or P.

In this project, the constraints work as follow:

- **Clock and Reset**: The 100 MHz system clock (clk) is tied to pin E3, the dedicated clock-capable pin on the Nexys A7. Two slide switches are repurposed as resets: reset maps to switch 13 and reset_clk to switch 14. These are the signals that drive the light and lights indicator outputs you can see in the RTL.
- **Buttons:** Three buttons are active. BTNC is mapped to **btn_pause**, the pause toggle. BTNL maps to **btn[0]** and BTNR to **btn[1]**, which are the physical paddle controls.
- **LED**: Seven standard LEDs (**LED[0]** through **LED[6]**) are mapped, plus two extra signals light and lights routed to LED positions 13 and 14. As established in the RTL, **LED[0]** and **LED[1]** show FIFO empty/full status, **LED[5:2]** show the FIFO count, and **LED[6]** indicates the paused state. The two RGB LEDs (**LED16** and **LED17**) are fully mapped with separate R, G, B pins, used to flash green on paddle hits and red on misses.
- **VGA:** All 14 VGA pins are active. The 12-bit rgb bus is split into four bits each for red, green, and blue, all routed to the VGA connector pins on bank 35. The **hsync** and **vsync** signals connect to pins B11 and B12.
- **Seven-Segment Display**: All 8 segment lines and all 8 digit-select anodes are constrained, enabling the full 8-digit score display to work.
- **UART:** Pins C4 and D4 map to *uart_rx* and *uart_tx* respectively, connecting to the onboard USB to serial bridge chip. This is what lets a terminal program on a computer communicate with the game over the USB cable.

(I/O EXPLANATION)

The FPGA board has two slide switches and three buttons. SW13 is FPGA reset: the screen turns off, the ball goes back to the middle, the paddle re-centers, and the score goes back to zero. SW14 is a "clock kill" switch that stops the FPGA's internal clock, which freezes everything. Each switch has a small green LED next to it that lights up while the switch is on, so it's easy to tell at a glance which one you flipped. The three buttons are the controls: BTNL moves the paddle left, BTNR moves the paddle right, and BTNC pauses the game. When the game is paused, SW 6 LED lights up and SW 13 LED lights up as well because the clock is paused. You can also play from a PC by typing into a serial terminal, A moves left, D moves right, S or space stops, and P pauses. The game has two displays. The VGA monitor shows the live game at 640×480. The 8-digit seven-segment display on the board shows the same score. Both displays read from the same score counter inside the chip, which adds 5 every time you hit the ball and resets to 0 if you miss, so the screen and the board always agree. Finally, the two RGB LEDs (LD16 & LD 17) are a quick status light, they flash green when you hit the ball and red when you miss.

(FSM EXPLANATION)

The two FSM in this project handle UART communication, one for *receiver* and one for *transmitter*. Both follow the same five-state sequence: **Idle**, **Start**, **Data Bits**, **Stop Bit**, and **Refresh**. The receiver sits in Idle waiting for the serial line to go low, which signals the start of a byte. It then samples the midpoint of each bit period using a counter that counts to 434 clock cycles, shifting each bit into **r_Byte** one at a time across 8 iterations. Once the stop bit is confirmed, it pulses **o_DV** for one cycle to signal that a valid byte is ready. The transmitter mirrors this in reverse when **i_DV** is asserted, it serializes the byte LSB first, driving **o_Serial_Data** for exactly one bit period per bit, then asserting **o_Sig_Done** when finished.

(FIFO EXPLANATION)

The FIFO is a 16-deep, 8-bit wide circular buffer that decouples the UART receiver from the game logic. Every time the receiver finishes a byte, *pong_top* checks if it's a recognized ASCII character and translates it into a semantic command byte which gets written into the

FIFO. The FIFO uses two wrap-around pointers, *wr_ptr* and *rd_ptr*, and a 5-bit occupancy counter to track how full it is.

On the consumption side, *pong_top* reads one command out of the FIFO every clock cycle as long as the game is not paused and the FIFO is not empty. This design means physical buttons and UART commands are treated identically by the game, and the FIFO ensures that even a burst of quickly typed keystrokes won't be lost, they queue up and are processed in order.

(MODELING STYLES EXPLANATION)

Behavioral modeling is the dominant style across the project. The FSMs in *receiver* and *transmitter* are written with the use of always @(posedge clk) blocks with case statements and the if/else logic which describes the system behavior. The FIFO, the score taking, the pause control and the UART handling that was in pong_top use this style too.

Data flop modeling is also used for the simple combinational logic like with the assign statements. This included the FIFO status signals like empty and full, which read the data from memory (dout) and the basic control signals like ctrl_btn.

Structural modeling was also used in pong_top, this is where the system was built by instantiating and connecting all the overall modules like the receiver, transmitter, FIFO, VGA sync, animation and the 7 segment. This shows the system hierarchy.

We have the global header file as well which was used in order to store UART timing, FIFO depth, ASCII values, the score tracking and the colors.

(EXPLANATION OF TESTBENCH STRATEGY)

We had multiple test benches that were used to verify the project, the 3 primary testbenches included “tb_master_pong”, “t_uart_cmd_fifo”, and “tb_uart_loopback”. Instead of just testing the final system, we had the individual subsystems getting tested first. It was more convenient to separate the testbenches before a whole system integration. These benches were

able to verify module all the implementations which included the UART communication, our FIFO queue behavior, our score logic and the reset behavior.

tb_uart_cmd_fifo

This bench instantiates **uart_cmd_fifo** in isolation and verifies reset behavior, correct data ordering through write-read cycles, and boundary conditions at the parameterized depth of 16. The full and empty flags are checked to assert and deassert at exactly the right occupancy levels. Testing the FIFO independently makes off by one pointer errors far easier to diagnose than inside a full system waveform.

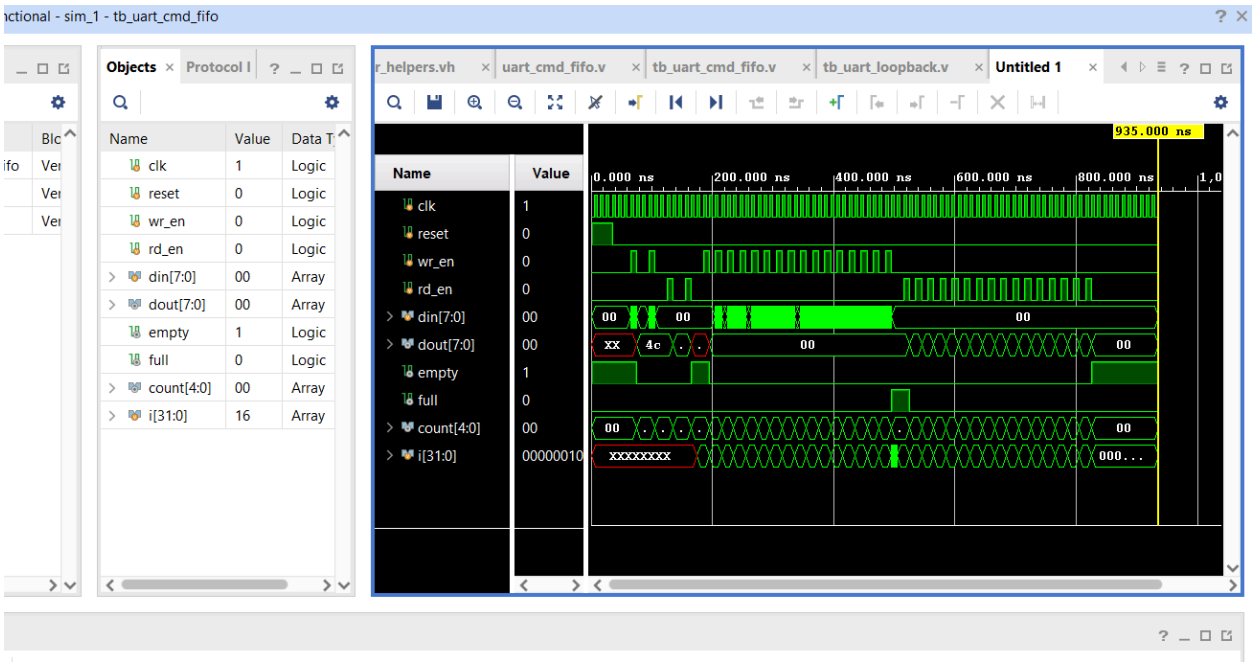
tb_uart_loopback

This bench connects transmitter and receiver in a direct loopback at 50 MHz, matching the divided clock domain used by the UART inside **pong_top**. The task **loopback_byte** serializes an 8N1 frame and confirms that received data matches what was sent, across the A, D, S, and U command bytes. Isolating baud-rate timing here prevents misconfiguration of **UART_DIV** from producing silent failures in the integration bench.

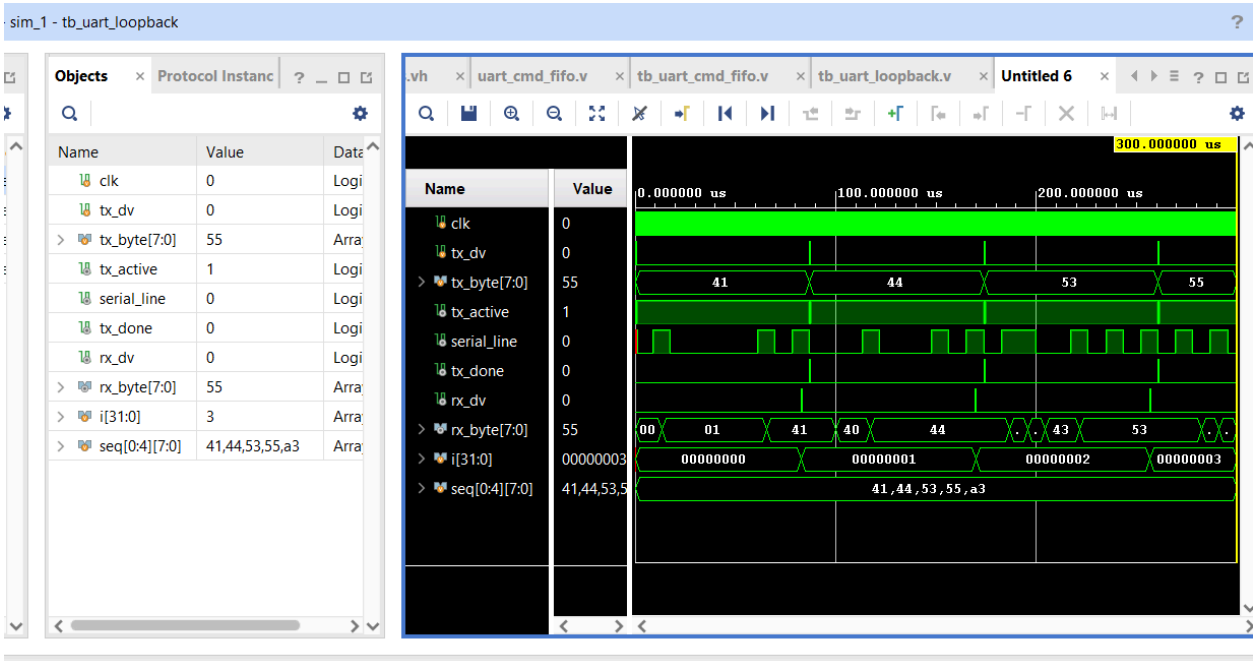
tb_master_pong

This bench instantiates the complete pong_top DUT at 100 MHz and includes **tb_master_helpers.vh** for full access to all shared tasks. Ten numbered tests progress from simple to complex. The startup test calls *apply_reset* and confirms that paused, *fifo_empty, fifo_count, score_bcd, uart_left,* and *uart_right* all return to their defaults. UART command tests serialize A, D, and S bytes and verify the resulting latch states. The pause test confirms that commands queue in the FIFO while paused is asserted and drain correctly after unpause. Score tests use force/release on *paddle_hit_pulse* and *miss_pulse* to bypass the VGA pipeline, verifying BCD increments, **LED16_G** on a hit, and **LED16_R** with a score clear on a miss. Runtime reset tests confirm that pulsing reset and *reset_clk* each restore the system to a clean state.

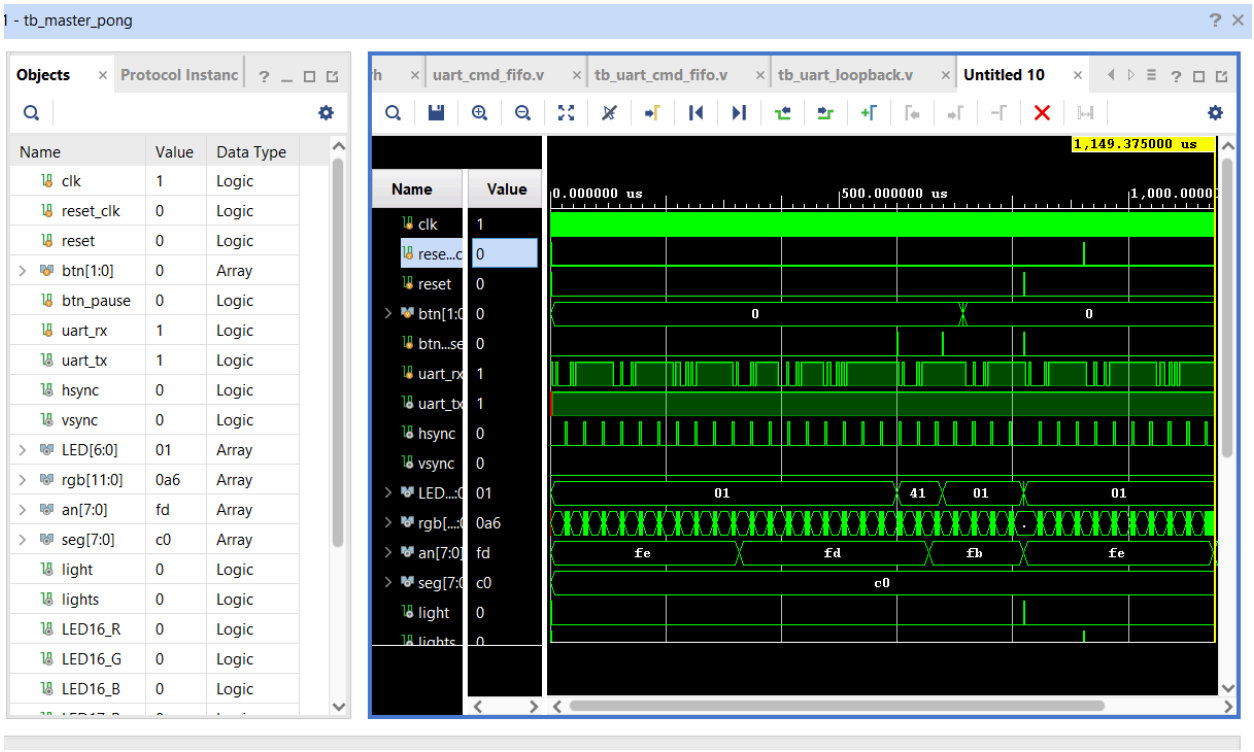
SCREENSHOTS OF SIMULATION



(image: FIFO simulation)



(image: UART loop communication)



(image: top module)

Brief conclusion

In general this project was able to successfully implement a functioning game of Pong on our Nexys A7, which integrated the VGA display alongside UART communication. In general all of these major components which included the display output, the inputs, the UART, and score tracking all served its purpose in order to be able to operate as a coordinated system. The project was able to achieve a stable gameplay that was able to correspond depending on the controls from either the FPGA itself or the connected PC. In general the various hardware interactions were sufficient enough to highlight the ultimate goal of this project.